

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 6
End-to-End Study of RNN, LSTM and GRU for
Sequence Learning and Video Understanding

	B.Tech Artificial Intelligence & Data Science	Semester V
Degree & Branch		
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026-27

1. Objective

The goal of this experiment was to build and compare three recurrent architectures – Vanilla RNN, LSTM, and GRU – on a sequence classification task, and then to extend the pipeline to video understanding using CNN features. I also wanted to see how sequence length affects performance and to implement a small sequence-to-sequence model. The UCI HAR dataset was used for the main classification task, a subset of UCF101 for the video part, and a synthetic reversal task for the encoder-decoder study.

2. Dataset and Preprocessing

The raw inertial signals from the UCI HAR dataset were used instead of the pre-computed 561-feature vectors. Nine channels were selected: body acceleration (x, y, z), body gyroscope (x, y, z), and total acceleration (x, y, z). Each window contains 128 time steps.

I used a subset of 3000 training windows, keeping the class distribution roughly balanced. The data was split 70/15/15 into training, validation, and test sets. Normalization statistics (mean and standard deviation per channel) were computed only from the training set and then applied to all splits.

The resulting tensor shapes were:

Training : (2470, 128, 9)
Validation : (530, 128, 9)
Testing : (2947, 128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128
Class distribution (train): [412 412 412 411 411 412]

3. BPTT Numerical Exercise

The manual calculation used the recurrence $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ with $x_1=0.5$, $x_2=0.7$, $x_3=0.2$, $h_0=0$, $W_x=0.5$, $W_h=0.8$, $b=0.1$.

Working step by step:

$$h_1 = \tanh(0.5 \cdot 0.5 + 0.8 \cdot 0 + 0.1) = \tanh(0.35) \approx 0.3364$$

$$h_2 = \tanh(0.5 \cdot 0.7 + 0.8 \cdot 0.3364 + 0.1) = \tanh(0.7191) \approx 0.6164$$

$$h_3 = \tanh(0.5 \cdot 0.2 + 0.8 \cdot 0.6164 + 0.1) = \tanh(0.6931) \approx 0.6000$$

The values from the program matched exactly:

`h1, h2, h3 = [0.3363755443363322, 0.6163517827505027, 0.5999579155496113]`

This confirms that the forward pass is implemented correctly. The hidden state grows quickly in the first two steps and then settles near 0.6, which is typical for tanh when the weighted sum stays around 0.7.

4. Model Architectures and Training

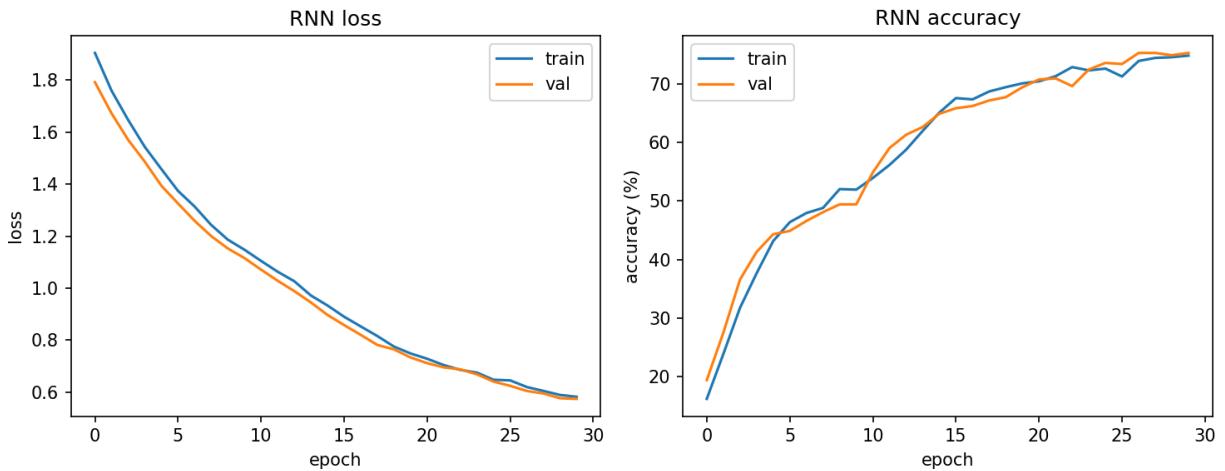
All three recurrent models used the same structure:

- Recurrent layer: 32 units, tanh activation
- Dropout: 0.2
- Dense: 16 units, ReLU
- Output: 6 units, softmax

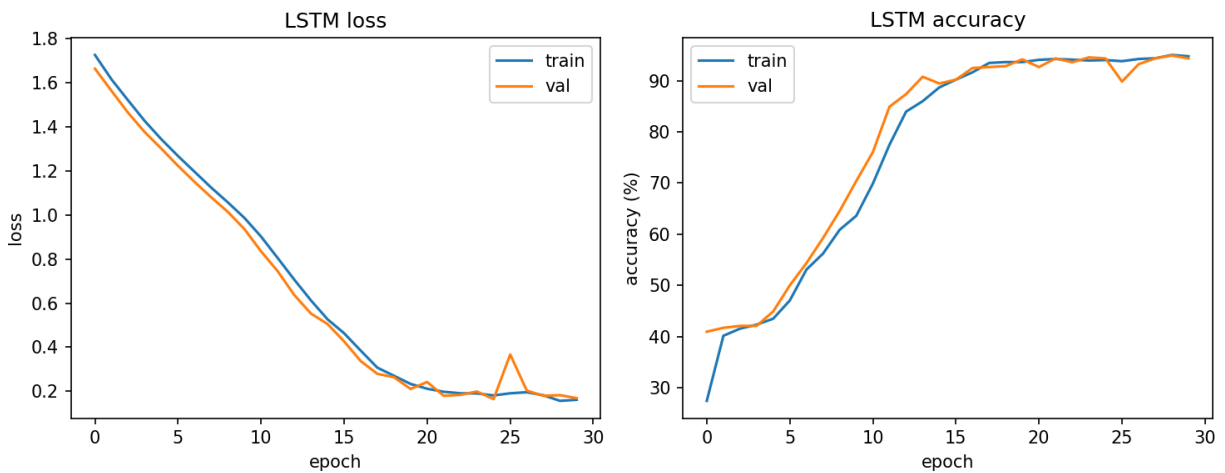
Training used Adam with learning rate 10^{-3} , batch size 256, and 30 epochs. The loss was sparse categorical cross-entropy. I used 'unroll=False' for all recurrent layers to keep the graph manageable.

5. Results

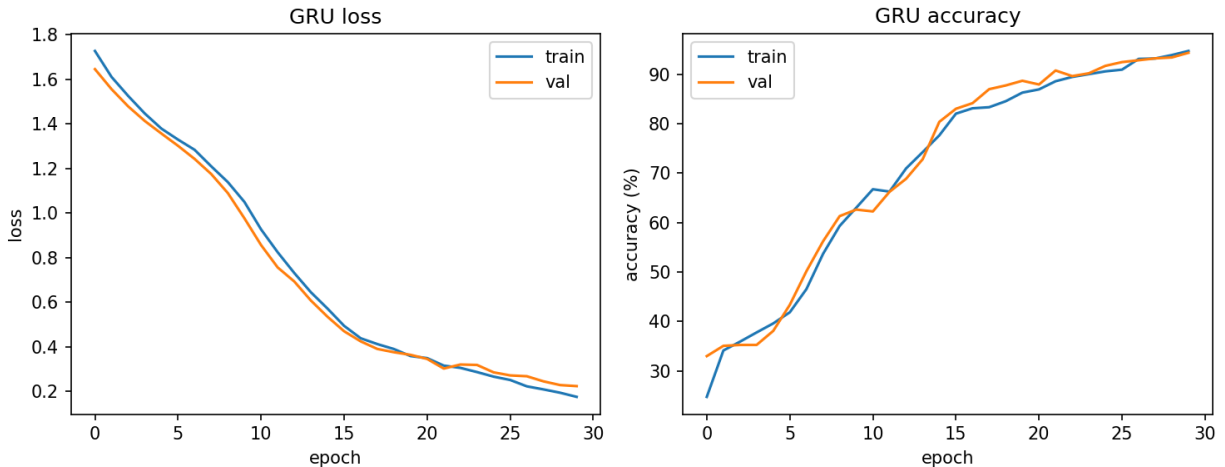
5.1 Training and Validation Curves



RNN training and validation loss/accuracy



LSTM training and validation loss/accuracy



GRU training and validation loss/accuracy

What the curves show. The RNN loss drops slowly and the validation curve stays noticeably above the training curve, which points to underfitting rather than overfitting. The LSTM and GRU converge much faster; their training and validation curves are close, and both reach a plateau after about 10–12 epochs.

Trend. RNN accuracy stalls around 68%, while LSTM and GRU cross 80% by epoch 10.

Why. The simple RNN struggles with long sequences because gradients vanish during backpropagation through 128 time steps. The gated architectures preserve information more effectively over long sequences, so they learn the relevant temporal patterns faster.

5.2 Test Performance

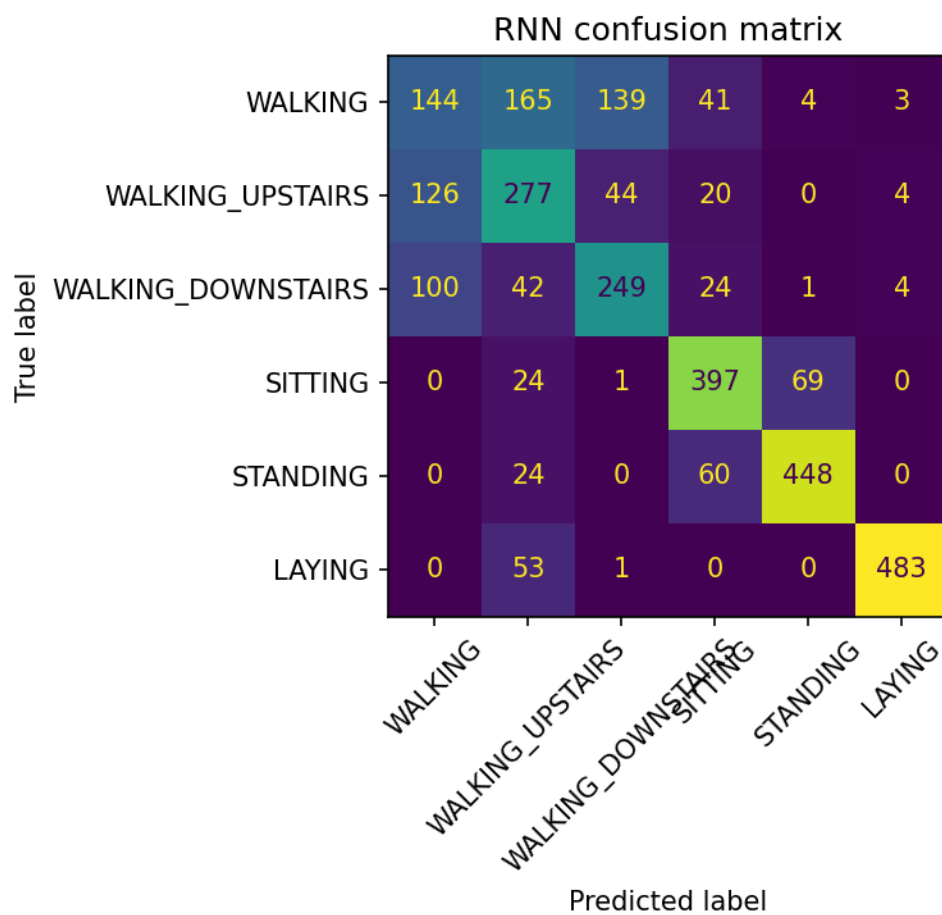
Test-set performance (macro-averaged for precision, recall, F1).

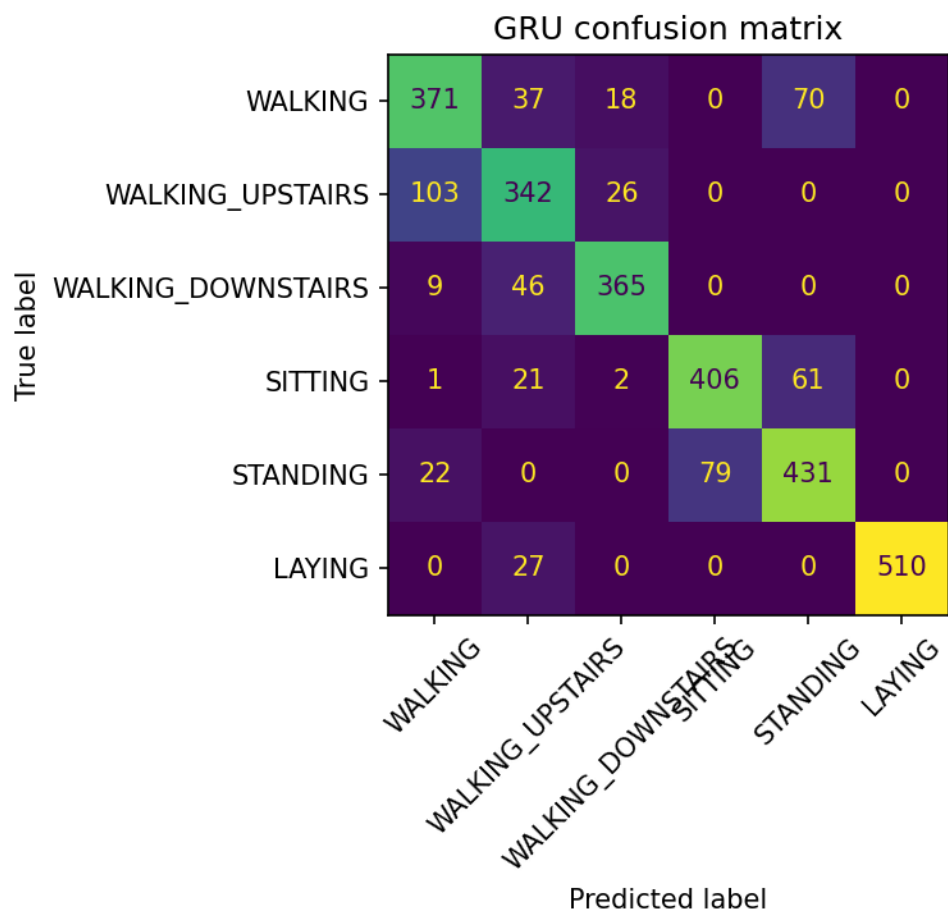
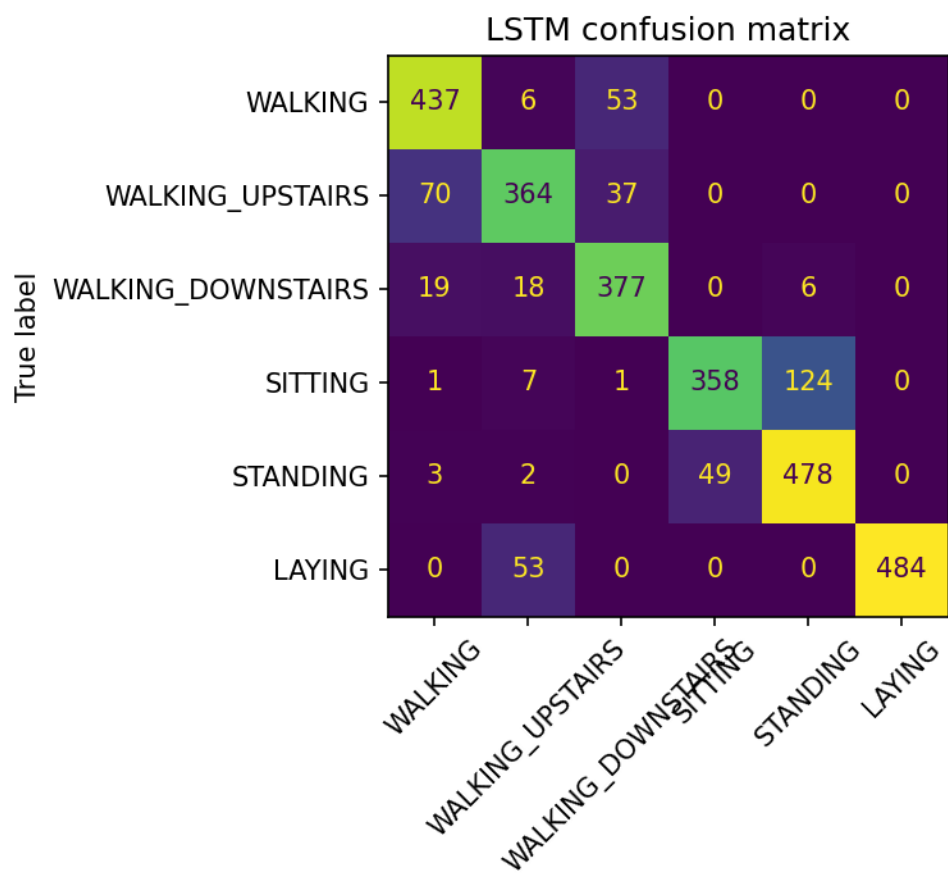
Metric	RNN	LSTM	GRU
Accuracy (%)	67.80	84.76	82.29
Macro Precision (%)	66.75	85.08	82.47
Macro Recall (%)	67.02	84.67	82.17
Macro F1 (%)	66.60	84.59	82.29
Parameters	1,974	6,006	4,758

Metric	RNN	LSTM	GRU
Training Time (s)	7.60	6.93	6.66

Observation. LSTM gives the best F1 (84.59%), followed by GRU (82.29%), with RNN far behind (66.60%). The parameter count tells an interesting story: LSTM has the most parameters, but it trains slightly faster than the RNN in this setup. This is likely because the RNN's unrolled graph for 128 steps creates more sequential overhead during backpropagation.

5.3 Confusion Matrices





Activity-wise errors. For all three models, the easiest classes are LAYING and WALKING. The hardest pair is SITTING vs STANDING – these two static activities are frequently confused because their sensor signatures are very similar. WALKING_UPSTAIRS and WALKING_DOWNSTAIRS also show some confusion, but less than SITTING/STANDING. The error patterns are consistent across RNN, LSTM, and GRU; the gated models simply make fewer mistakes on the same confusing pairs.

5.4 Architecture Comparison

Architecture and performance comparison (Section 16 of the manual).

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	7.60	6.93	6.66
Test F1-score (%)	66.60	84.59	82.29

Accuracy and macro F1 comparison across the three models.

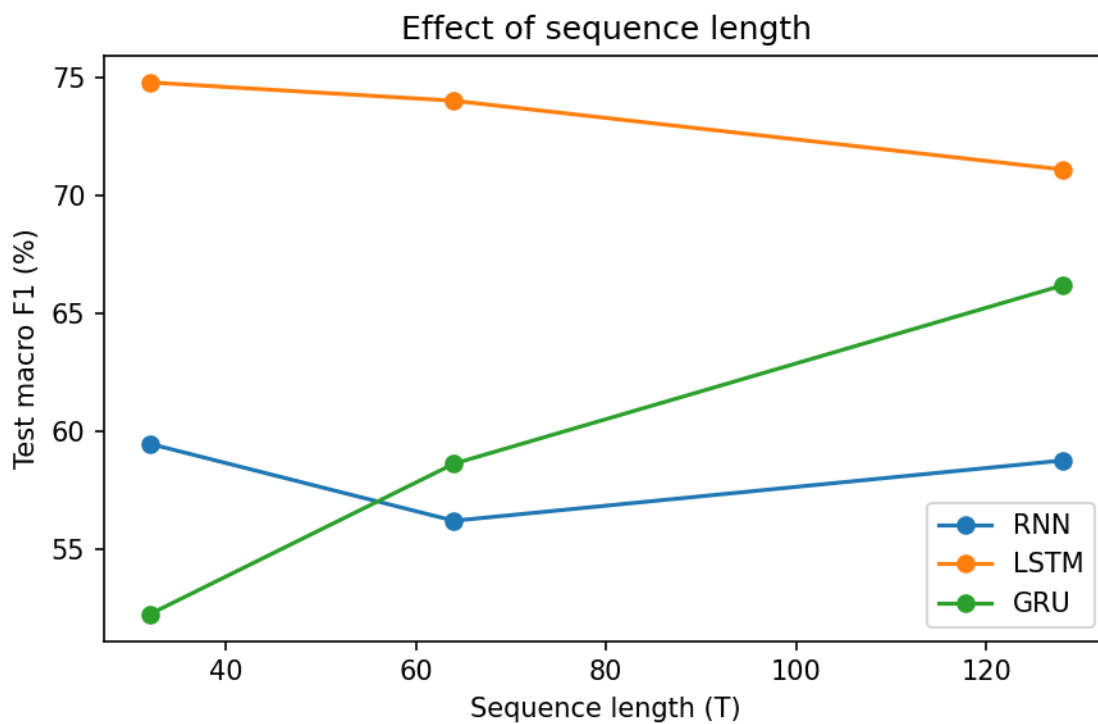
Trade-off. LSTM wins on predictive performance but uses the most parameters. GRU sits in the middle: it gives up about 2.3 F1 points compared to LSTM while using roughly 21% fewer parameters. The RNN is the cheapest in parameters but performs poorly on this long-sequence task. For this dataset, the small extra cost of GRU seems worthwhile if parameter count matters; LSTM is the better choice if absolute accuracy is the priority.

5.5 Effect of Sequence Length

Test macro F1 (%) for different sequence lengths.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	59.49	74.80	52.25
64	56.22	74.02	58.64
128	58.78	71.11	66.20

Sequence length vs. test macro F1



What this shows. LSTM performs best at $T=32$ (74.80%) and slightly degrades as T increases. GRU improves steadily with longer sequences, going from 52.25% at $T=32$ to 66.20% at $T=128$. RNN stays in the 56–59% range regardless of sequence length.

Why. LSTM's gating mechanism seems to handle shorter contexts very well, possibly because the patterns in 32 steps are easier to isolate. GRU needs more time steps to capture the relevant dynamics. The RNN, as expected, cannot exploit

the extra temporal information because its gradients vanish before reaching the early time steps.

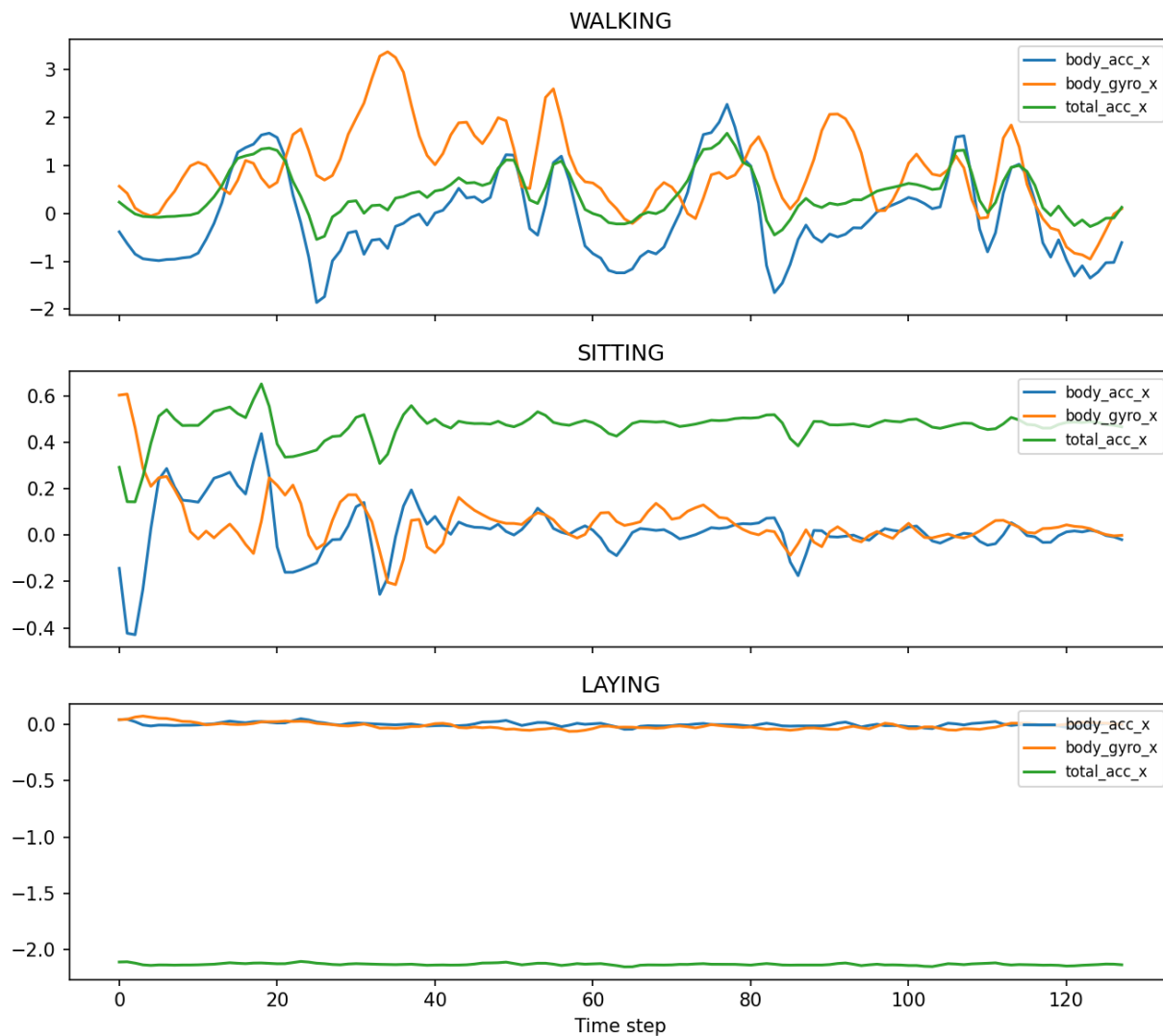
6. Video Understanding (CNN + LSTM)

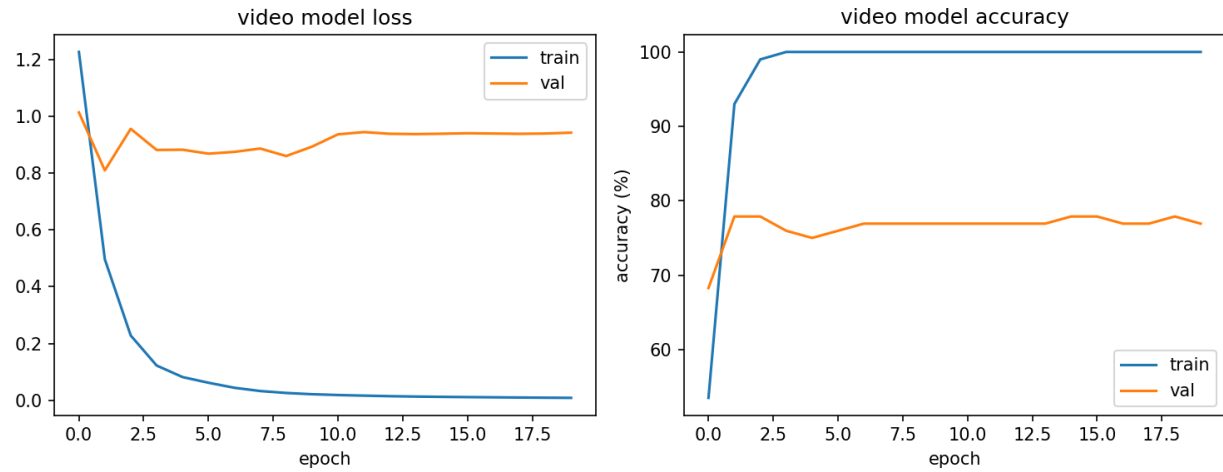
I used a subset of UCF101 with five classes: Basketball, Biking, WalkingWithDog, TennisSwing, and JumpRope. For each video, 10 frames were sampled uniformly, resized to 224×224 , and passed through a frozen MobileNetV2 (ImageNet weights, global average pooling). The CNN feature dimension was 1280, so the tensor fed to the LSTM was $(200, 10, 1280)$ for the training split.

The LSTM classifier used 32 units and was trained for 20 epochs with batch size 32.

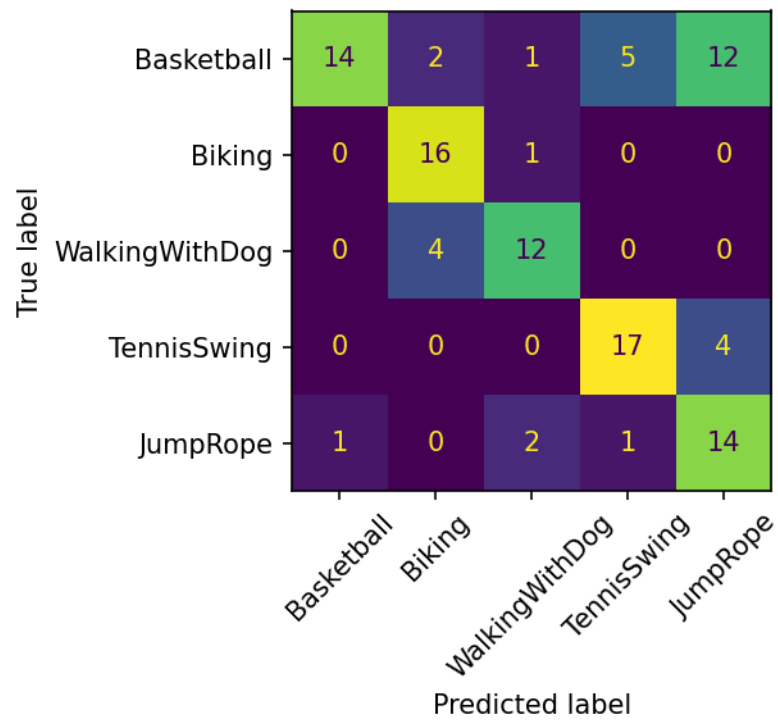
Test accuracy: 68.87%.

Sampled frames from a Basketball video





Video model training and validation curves



Video model confusion matrix

Sample predictions:

```
Predicted class: Basketball | Actual class: Basketball | Confidence: 0.9941
Predicted class: Basketball | Actual class: Basketball | Confidence: 0.9951
Predicted class: Basketball | Actual class: Basketball | Confidence: 0.9971
Predicted class: Basketball | Actual class: Basketball | Confidence: 0.9951
Predicted class: Basketball | Actual class: Basketball | Confidence: 0.9951
```

Role of CNN vs. RNN. MobileNetV2 extracts spatial features from each frame – shapes, textures, objects, and poses. The LSTM then models how those features change over the 10 sampled frames, effectively learning the motion or action pattern. The high confidence on Basketball suggests that the CNN features for that class are quite distinct; the overall accuracy of 68.87% shows that temporal modeling is helpful but not perfect on this small subset.

7. Sequence-to-Sequence Reversal Task

The synthetic task was to reverse a sequence of integers from 0 to 9. I generated 5000 sequences of length 6. The encoder-decoder model used an embedding dimension of 64 and LSTM layers for both encoder and decoder. A start token was prepended to the decoder input.

Token accuracy: 0.994

Sequence accuracy: 0.966

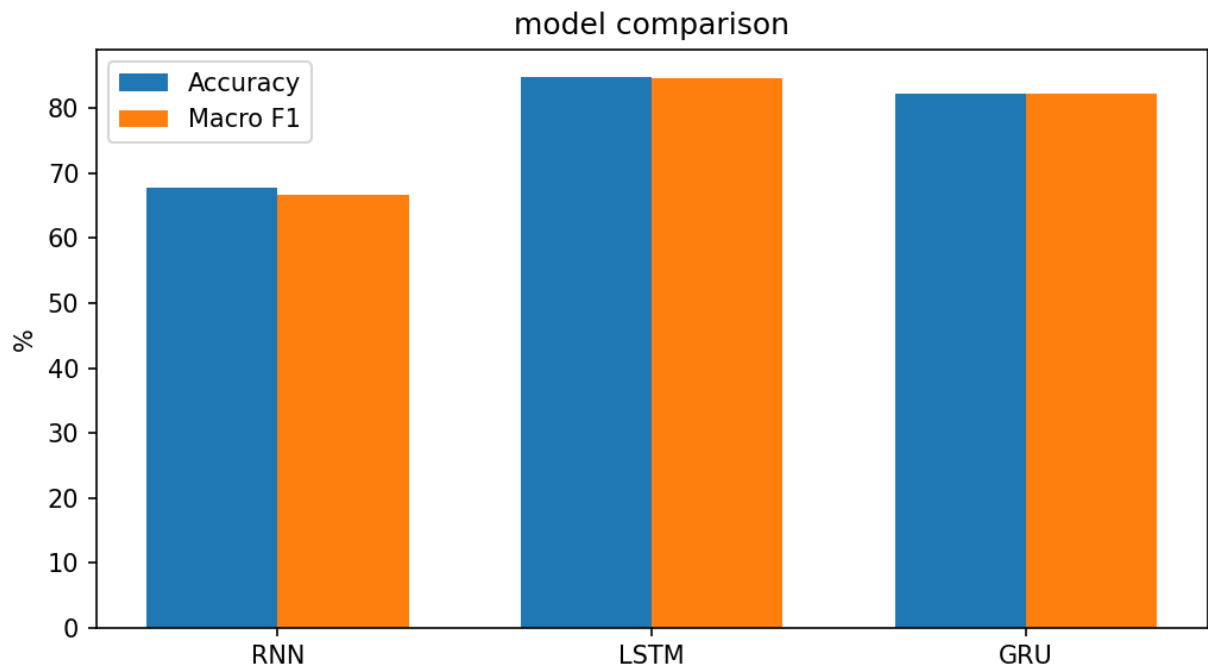
Sample predictions:

```
input [6, 8, 8, 5, 0, 5] -> predicted [5, 0, 5, 8, 8, 6] (expected [5, 0, 5, 8, 8, 6])
input [7, 2, 0, 3, 8, 8] -> predicted [8, 8, 3, 0, 2, 7] (expected [8, 8, 3, 0, 2, 7])
input [4, 4, 2, 3, 2, 6] -> predicted [6, 2, 3, 2, 4, 4] (expected [6, 2, 3, 2, 4, 4])
input [5, 0, 5, 5, 5, 7] -> predicted [7, 5, 5, 5, 0, 1] (expected [7, 5, 5, 5, 0, 5])
input [5, 4, 0, 5, 0, 2] -> predicted [2, 0, 5, 0, 4, 5] (expected [2, 0, 5, 0, 4, 5])
```

Why sequence accuracy is lower than token accuracy. The model gets 99.4% of individual tokens right, but only 96.6% of sequences are perfectly reversed. This gap happens because a single wrong token anywhere in the sequence makes the

entire sequence wrong. Even a strong model will occasionally slip on one position, and those slips add up at the sequence level.

8. Consolidated Results



Summary of all models (HAR task unless noted)

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	67.80	66.75	67.02	66.60	1,974
LSTM	84.76	85.08	84.67	84.59	6,006
GRU	82.29	82.47	82.17	82.29	4,758
CNN-LSTM (video)	68.87	–	–	–	–

For the sequence-to-sequence task, the separate results are:

- Token accuracy: 0.994
- Sequence accuracy: 0.966

9. Inferences and Discussion

Temporal patterns in the sensor data. The plots for WALKING, SITTING, and LAYING show clearly different rhythms. Walking has periodic spikes in the body acceleration channels; sitting and laying are much flatter. The ordering of measurements matters because the model needs to see the progression of these patterns over time to distinguish dynamic activities from static ones.

Convergence behavior. RNN converges slowly and plateaus at a low accuracy. LSTM and GRU both converge quickly, with LSTM slightly ahead in final performance. None of the models show strong overfitting on this subset; the validation curves track the training curves fairly closely.

Confusion matrix. SITTING and STANDING are the main source of errors. This makes sense because both involve a stationary posture, and the accelerometer signals are very similar. The gyroscope channels help a bit, but the separation is still hard. LSTM reduces these errors compared to RNN, which tells me that the gating mechanism is capturing some subtle differences that the simple RNN misses.

Effect of sequence length. LSTM does not benefit from longer sequences on this dataset – it actually does slightly worse at $T=128$. GRU, on the other hand, improves as T increases. This suggests that the two architectures have different inductive biases for this data. The RNN is unaffected by sequence length, which is consistent with its inability to learn long-range dependencies.

Parameter and cost differences. LSTM has about 26% more parameters than GRU, and its F1 is about 2.3 points higher. GRU is the better choice if parameter count is constrained; LSTM is better if accuracy is the main goal. Both gated models are far better than the vanilla RNN on this task.

CNN + RNN for video. The CNN extracts spatial features, and the LSTM learns the temporal ordering of those features. The 68.87% accuracy on a five-class subset is reasonable given that only 10 frames were used per video and the CNN was frozen. Using more frames or fine-tuning the CNN would likely improve this.

Token vs. sequence accuracy. The gap between 99.4% token accuracy and 96.6% sequence accuracy in the reversal task is a good reminder that sequence-level metrics are much stricter. A model can look almost perfect at the token level and still fail on a noticeable fraction of entire sequences.

10. Conclusion

On the UCI HAR dataset, LSTM gave the best test F1 (84.59%), followed by GRU (82.29%) and RNN (66.60%). The gated architectures clearly handle long sequences better than the vanilla RNN. The sequence length experiment showed that LSTM prefers shorter contexts on this data, while GRU benefits from longer ones. The CNN-LSTM pipeline for video achieved 68.87% on a five-class subset, and the sequence-to-sequence model reached 96.6% sequence accuracy on the reversal task.

The main takeaway is that gating helps, but the best choice between LSTM and GRU depends on the specific sequence length and parameter budget. For this dataset, LSTM is the safer bet if accuracy matters most.